Работа с каталогами

Цель работы
Научиться создавать безопасную тестовую структуру каталогов в рабочей папке проекта, проверять существование и тип пути, получать список файлов и подпапок, различать файлы и каталоги и подсчитывать объекты. Закрепить работу с pathlib.Path без перехода к копированию, перемещению и переименованию.

Среда выполнения: Python 3.x, Visual Studio Community. Все действия выполняются только внутри рабочей папки проекта. Продолжительность — 2 академических часа.

1. Связь с предыдущими практическими работами

В предыдущей работе путь использовался для исследования одного файла: проверялось существование, определялись имя, расширение, размер и другие свойства. Теперь объектом исследования становится каталог — папка, которая может содержать другие каталоги и файлы.

Для работы снова используется pathlib и объект Path. Новыми основными действиями будут создание каталогов через mkdir(), проверка is_dir() и просмотр содержимого через iterdir().

Граница темы
В этой работе разрешено создавать только собственные тестовые каталоги и файлы в рабочей папке проекта. Не выполняются копирование, перемещение, переименование и удаление — эти операции относятся к следующей практической работе.

2. Основные понятия

Конструкция	Назначение	Пример
Path.cwd()	Возвращает текущую рабочую папку.	C:\...\DirectoryPractice
mkdir()	Создаёт каталог.	folder.mkdir()
exist_ok=True	Разрешает повторный запуск, если каталог уже существует.	mkdir(exist_ok=True)
parents=True	Разрешает создать недостающие родительские каталоги.	mkdir(parents=True, exist_ok=True)
is_dir()	Проверяет, что путь указывает на каталог.	True / False

iterdir()	Перебирает непосредственное содержимое каталога.	for item in folder.iterdir():

3. Подготовка проекта

Создайте отдельный Python-проект с понятным именем, например DirectoryPractice. Весь код этой практической можно выполнять в одном файле main.py, последовательно дополняя его.

Сначала выведите текущую рабочую папку. Это позволит понять, где именно Python будет создавать тестовые каталоги.

```
Определение рабочей папки
from pathlib import Path

work_dir = Path.cwd()

print("Рабочая папка:")
print(work_dir)
```

Path.cwd() возвращает текущую рабочую папку. В ней будет создан отдельный учебный каталог. Не копируйте путь из примера: на каждом компьютере он может отличаться.

```
ЗАДАНИЕ 1. Зафиксировать рабочую папку
1. Создайте или откройте проект DirectoryPractice.
2. Добавьте импорт Path.
3. Выведите Path.cwd().
4. Запустите программу.
5. Сделайте скриншот консоли, на котором виден полный путь.

В отчёте под скриншотом напишите 1–2 предложения: что показывает Path.cwd() и почему все дальнейшие учебные объекты создаются именно внутри этой папки.
```

4. Создание первого каталога — mkdir()

Для создания каталога у объекта Path используется метод mkdir(). Сначала сформируем путь, а затем создадим папку.

Пример: один каталог

```
from pathlib import Path

base = Path.cwd() / "demo_catalog"
base.mkdir(exist_ok=True)

print("Каталог подготовлен:", base)
```

Здесь выражение Path.cwd() / "demo_catalog" формирует путь к будущей папке. Метод mkdir() создаёт её.

Что означает exist_ok=True
Если каталог demo_catalog уже существует после предыдущего запуска, exist_ok=True разрешает продолжить работу без ошибки. Это удобно в учебной программе, которую запускают несколько раз.

Если убрать exist_ok=True и попытаться повторно создать существующий каталог, Python может сообщить об ошибке FileExistsError.

ЗАДАНИЕ 2. Создать собственный базовый каталог
Создайте внутри рабочей папки каталог practice40.

Требования:
• путь сформировать через Path.cwd();
• использовать mkdir(exist_ok=True);
• после создания вывести полный путь;
• открыть Проводник и убедиться, что practice40 действительно появился.

Сделайте скриншот Проводника или консоли. В пояснении укажите, для чего использован exist_ok=True.

5. Проверка каталога: exists() и is_dir()

Метод exists() уже знаком по работе с файлами: он сообщает, существует ли объект по указанному пути. Но одного exists() недостаточно — путь может вести как к файлу, так и к папке.

Для проверки каталога используется is_dir().

Проверка каталога

```
base = Path.cwd() / "demo_catalog"

print("Существует:", base.exists())
print("Это каталог:", base.is_dir())
```

Пример результата
Существует: True
Это каталог: True

Сравните с is_file(): is_file() проверяет обычный файл, а is_dir() — каталог.

ЗАДАНИЕ 3. Проверить practice40
Для каталога practice40 выведите результаты exists(), is_dir() и is_file().

Ожидаемая логика:
• exists() — True;
• is_dir() — True;
• is_file() — False.

Не копируйте эти значения в отчёт заранее: сначала выполните код и зафиксируйте фактический результат. Кратко объясните, почему один и тот же путь даёт разные результаты is_dir() и is_file().

6. Создание вложенной структуры каталогов

Обычно программа работает не с одной папкой, а со структурой. Например, отдельные каталоги могут хранить документы, журналы и архивные данные.

Рассмотрим учебную структуру:

Пример структуры
```
demo_catalog
├── docs
├── logs
└── archive
```

Код для создания структуры
```
from pathlib import Path

base = Path.cwd() / "demo_catalog"
docs = base / "docs"
logs = base / "logs"
archive = base / "archive"

docs.mkdir(parents=True, exist_ok=True)
logs.mkdir(parents=True, exist_ok=True)
archive.mkdir(parents=True, exist_ok=True)
```

```
print("Структура каталогов создана")
```

Параметр parents=True разрешает создать недостающие родительские папки. Например, если demo_catalog ещё не существует, вызов docs.mkdir(parents=True, exist_ok=True) сможет сначала создать demo_catalog, а затем docs.

Важно
parents=True не означает «создать много случайных папок». Он только разрешает Python создать недостающие части указанного пути.

ЗАДАНИЕ 4. Построить структуру practice40
Создайте внутри practice40 три каталога:
• materials;
• reports;
• logs.

Используйте Path и mkdir(parents=True, exist_ok=True). После запуска откройте practice40 в Проводнике и сделайте скриншот структуры.

В отчёте поясните, чем base / "materials" отличается от обычной текстовой строки "materials".

7. Подготовка тестовых файлов внутри каталогов

Чтобы было что исследовать, создадим несколько небольших файлов. Способ записи уже изучался ранее, поэтому здесь он используется как вспомогательное действие.

Пример наполнения demo_catalog

```
from pathlib import Path

base = Path.cwd() / "demo_catalog"
docs = base / "docs"
logs = base / "logs"

docs.mkdir(parents=True, exist_ok=True)
logs.mkdir(parents=True, exist_ok=True)

with open(docs / "report.txt", "w", encoding="utf-8") as file:
    file.write("Учебный отчёт\n")

with open(docs / "notes.txt", "w", encoding="utf-8") as file:
    file.write("Заметки по работе\n")
```

```
with open(logs / "app.log", "w", encoding="utf-8") as file:
    file.write("Program started\n")
```

Обратите внимание: в open() передаётся не просто имя файла, а путь вида docs / "report.txt". Поэтому report.txt создаётся именно внутри docs.

ЗАДАНИЕ 5. Наполнить practice40 тестовыми файлами
Создайте программным способом:
• materials\theory.txt — минимум две строки;
• reports\result.txt — минимум две строки;
• logs\session.log — минимум одну строку.

Не создавайте файлы вручную через Проводник: в этой части нужно использовать Python.

После запуска откройте practice40 и убедитесь, что каждый файл находится в нужной подпапке. Сделайте скриншот одной из подпапок с созданным файлом.

8. Получение содержимого каталога — iterdir()

Метод iterdir() позволяет получить объекты, которые непосредственно находятся внутри указанного каталога.

Простой пример
```
base = Path.cwd() / "demo_catalog"

for item in base.iterdir():
    print(item.name)
```

Пример результата
```
docs
logs
archive
```

Переменная item на каждой итерации содержит объект Path. Свойство item.name выводит только имя, а print(item) показал бы путь.

Что важно помнить
iterdir() показывает только непосредственное содержимое указанного каталога. Он не открывает автоматически все вложенные папки.

ЗАДАНИЕ 6. Вывести содержимое practice40
Переберите practice40 через iterdir() и выведите имена всех объектов.

Сравните консольный вывод со структурой в Проводнике. В отчёте напишите, совпало ли

количество объектов верхнего уровня и какие именно имена были получены.

9. Различение файлов и каталогов

При просмотре содержимого полезно не просто выводить имя, а понимать тип объекта. Для этого применяются is_file() и is_dir().

```
Пример классификации
base = Path.cwd() / "demo_catalog"

for item in base.iterdir():
    if item.is_file():
        print("[ФАЙЛ]", item.name)
    elif item.is_dir():
        print("[КАТАЛОГ]", item.name)
```

Для верхнего уровня demo_catalog результат будет состоять в основном из каталогов. Если тем же способом проверить docs, будут обнаружены файлы report.txt и notes.txt.

```
Похожий пример для вложенного каталога
docs = Path.cwd() / "demo_catalog" / "docs"

for item in docs.iterdir():
    if item.is_file():
        print("[ФАЙЛ]", item.name)
    elif item.is_dir():
        print("[КАТАЛОГ]", item.name)
```

ЗАДАНИЕ 7. Классифицировать объекты practice40
1. Выведите содержимое practice40 с метками [ФАЙЛ] и [КАТАЛОГ].
2. Отдельно выведите содержимое materials с такими же метками.
3. Сделайте скриншот консоли.
4. В пояснении укажите, почему theory.txt определяется как файл, а materials — как каталог.

10. Подсчёт файлов и каталогов

Теперь добавим небольшую обработку результатов. Для этого достаточно уже знакомых переменных-счётчиков, цикла for и условий.

```
Пример подсчёта
base = Path.cwd() / "demo_catalog"

file_count = 0
folder_count = 0
```

```
for item in base.iterdir():
    if item.is_file():
        file_count += 1
    elif item.is_dir():
        folder_count += 1

print("Файлов:", file_count)
print("Каталогов:", folder_count)
```

Если на верхнем уровне находятся только docs, logs и archive, количество файлов будет 0, а каталогов — 3. Но внутри docs значения будут другими.

ЗАДАНИЕ 8. Подсчитать объекты
Подсчитайте:
• количество каталогов непосредственно внутри practice40;
• количество файлов непосредственно внутри materials;
• количество файлов непосредственно внутри reports.

Результаты внесите в таблицу наблюдений в отчёте. Если значения отличаются от ожидаемых, сначала проверьте фактическое содержимое папок.

11. Полный учебный пример

Ниже собраны основные конструкции этой работы в одной программе. Используйте пример как ориентир по последовательности действий, но итоговое задание выполняется с другой структурой и другими именами.

Полный пример «Каталог проекта»

```
from pathlib import Path

base = Path.cwd() / "demo_catalog"
docs = base / "docs"
logs = base / "logs"
archive = base / "archive"

docs.mkdir(parents=True, exist_ok=True)
logs.mkdir(parents=True, exist_ok=True)
archive.mkdir(parents=True, exist_ok=True)

with open(docs / "report.txt", "w", encoding="utf-8") as file:
    file.write("Отчёт готов\n")

with open(docs / "notes.txt", "w", encoding="utf-8") as file:
    file.write("Заметки\n")
```

```
with open(logs / "app.log", "w", encoding="utf-8") as file:
    file.write("Запуск программы\n")

print("Путь:", base)
print("Существует:", base.exists())
print("Это каталог:", base.is_dir())

print("\nСодержимое demo_catalog:")
for item in base.iterdir():
    if item.is_file():
        print("[ФАЙЛ]", item.name)
    elif item.is_dir():
        print("[КАТАЛОГ]", item.name)

print("\nСодержимое docs:")
file_count = 0

for item in docs.iterdir():
    if item.is_file():
        print("[ФАЙЛ]", item.name)
        file_count += 1

print("Файлов в docs:", file_count)
```

```
Пример результата
Путь: C:\...\DirectoryPractice\demo_catalog
Существует: True
Это каталог: True

Содержимое demo_catalog:
[КАТАЛОГ] docs
[КАТАЛОГ] logs
[КАТАЛОГ] archive

Содержимое docs:
[ФАЙЛ] report.txt
[ФАЙЛ] notes.txt
Файлов в docs: 2
```

В вашем выводе полный путь будет другим. Порядок объектов, возвращаемых iterdir(), также не следует считать гарантированным — ориентируйтесь на состав, а не на точную последовательность строк.

12. Итоговая самостоятельная работа

Итоговая программа должна использовать те же принципы, что и учебный пример, но структура и имена объектов отличаются. Готовое решение целиком не приводится.

ЗАДАНИЕ 9. Программа «Анализатор рабочей папки»
Создайте структуру workspace:

```
workspace
├── materials
│   └── theory.txt
├── reports
│   └── result.txt
├── logs
│   └── session.log
└── temp
```

Требования к программе:
1. Сформировать путь workspace внутри Path.cwd().
2. Создать все каталоги средствами Python.
3. Создать три указанных тестовых файла средствами Python.
4. Вывести полный путь к workspace.
5. Проверить exists() и is_dir().
6. Вывести непосредственное содержимое workspace через iterdir().
7. Для каждого объекта вывести [ФАЙЛ] или [КАТАЛОГ].
8. Отдельно вывести содержимое materials и reports.
9. Подсчитать количество файлов в materials и reports.
10. Вывести понятный итоговый текст с полученными значениями.

Используйте примеры из разделов 4–11 как ориентир, но не заменяйте имена из задания именами demo_catalog, docs или archive.

13. Небольшое самостоятельное изменение

Чтобы итоговая работа не сводилась к точному повторению примера, добавьте один собственный элемент структуры.

ЗАДАНИЕ 10. Дополнить структуру
Создайте внутри workspace ещё один каталог по своему выбору, например input, notes или backup_data. Внутри него создайте минимум один текстовый файл.

После изменения программа должна обнаружить новый каталог при просмотре workspace. Дополнительно выведите содержимое нового каталога и укажите количество файлов в нём.

В отчёте поясните, какое изменение внесено и какой фрагмент вывода подтверждает результат.

14. Алгоритм выполнения итоговой части

Шаг	Действие
1	Сформировать путь к workspace внутри Path.cwd().
2	Создать базовый каталог и вложенные папки через mkdir().
3	Создать тестовые файлы внутри нужных подпапок.
4	Проверить базовый каталог через exists() и is_dir().
5	Перебрать содержимое через iterdir(), различить is_file()/is_dir() и подсчитать объекты.
6	Добавить собственный каталог/файл, повторить проверку и зафиксировать результат в отчёте.

15. Если результат отличается от ожидаемого

Проблема	Что проверить
При повторном запуске возникает FileExistsError	Используйте mkdir(exist_ok=True), чтобы повторный запуск не считался ошибкой, если каталог уже существует.
При создании файла возникает FileNotFoundError	Сначала создайте родительский каталог. Для вложенной структуры используйте mkdir(parents=True, exist_ok=True).
iterdir() вызывает ошибку	Перед iterdir() проверьте exists() и is_dir(). Перебирать нужно существующий каталог.
Файл создан не в той папке	Выведите Path.cwd() и проверьте путь, переданный в open(). Файл создаётся по указанному пути, а не обязательно рядом с main.py.
Файл не виден в списке верхнего уровня	iterdir() показывает только непосредственное содержимое. Для вложенной папки выполните отдельный перебор этой папки.

Количество объектов не совпадает	Проверьте фактическое содержимое папки и увеличивайте счётчик только внутри нужной ветви is_file() или is_dir().

16. Что должно быть в Word-отчёте

Основной сдаваемый результат — Word-документ. Один файл main.py без отчёта не считается полностью оформленной практической работой.

Рекомендуемое имя файла: ПР40_Фамилия_Группа.docx.

Раздел отчёта	**Что должно быть в Word**
Тема и цель	Название ПР и кратко сформулированная цель работы.
Рабочая папка	Скриншот вывода Path.cwd() и 1–2 предложения о рабочей папке.
Структура practice40	Скриншот Проводника с materials, reports, logs и пояснение структуры.
Проверка типа	Скриншот exists(), is_dir(), is_file() и краткое объяснение различий.
Содержимое и классификация	Скриншот консоли с [ФАЙЛ] / [КАТАЛОГ] и пояснение полученного результата.
Итоговая программа	Полный код программы «Анализатор рабочей папки» текстом с сохранением отступов.
Итоговый результат	Скриншот workspace и итогового вывода программы: классификация и подсчёты.
Самостоятельное изменение	Скриншот и пояснение собственного дополнительного каталога и файла.
Таблица наблюдений	Заполненная таблица фактических результатов из раздела 17.
Контрольные вопросы	Ответы на 10 вопросов из раздела 18.
Вывод	3–5 предложений: какие каталоги были созданы, чем is_dir() отличается от is_file(),

	как использован iterdir() и какой результат получен.

Правило для скриншотов
Каждый скриншот подпишите в формате «Рисунок N — ...». Под каждым рисунком напишите 1–3 предложения своими словами: что показано, какое действие выполнено и какой результат подтверждает изображение.

Код итоговой программы вставьте в Word текстом с сохранением отступов. Скриншот редактора не заменяет текст кода.

17. Таблица наблюдений

Заполните таблицу по фактическим результатам своей программы. Не копируйте числа из примеров.

Наблюдение	Фактическое значение	Краткое пояснение
Содержимое верхнего уровня practice40		
Содержимое materials		
Содержимое reports		
Количество каталогов в practice40		
Количество файлов в materials		
Количество файлов в reports		
Дополнительный каталог		

18. Вопросы для проверки понимания

1. Чем каталог отличается от обычного файла?
2. Для чего используется mkdir()?
3. Что изменяет параметр exist_ok=True?
4. Для чего нужен параметр parents=True?
5. Что проверяет is_dir()?

6. Чем is_dir() отличается от is_file()?
7. Что возвращает iterdir() на базовом уровне?
8. Почему iterdir() для workspace не выводит автоматически файлы, находящиеся внутри materials?
9. Зачем передавать в open() выражение materials / "theory.txt", а не только имя файла?
10. Почему копирование, перемещение и переименование не выполняются в этой практической работе?

19. Критерии готовности

Проверка	Да / нет
Структура workspace создана только в рабочей папке	
Три тестовых файла созданы в нужных подпапках	
Содержимое каталогов получено через iterdir()	
Файлы и каталоги различаются через is_file() / is_dir()	
Подсчёт файлов и каталогов выполнен	
Самостоятельно добавлен дополнительный каталог и файл	
В отчёте не менее 5 скриншотов с пояснениями	
В Word вставлен итоговый код текстом	
Заполнена таблица наблюдений	
Даны ответы на 10 вопросов	
Написан краткий вывод	